BRAINWARE UNIVERSITY

BRAINWARE UNIVERSITY

PYTHON PROJECT-BASED LEARNING REPORT (WEEK 4)

1. Basic Information

Title of the Project:	**Password Strength Checker & Email Validation System** (Program 1 & 2)
Name(s) of Student(s):	**Soumyadwip Das (BWU/BTS/25/169)** **Ankita Paul (BWU/BTS/25/180)** **Sonia Naskar (BWU/BTS/25/195)** **Joyeta Mondal (BWU/BTS/25/214)** **Sayantan Bharati (BWU/BTS/25/503)**
Programme & Semester:	**B.Tech CSE & 3rd Semester**
Course Name & Code:	**Python Programming Lab (BES09013)**
Name of the Guide/Supervisor:	**MR. PRANAB GHARAI & MR. INDRANIL DAS**
Project Date	**31/07/2026**

2. Overview

This week's lab includes two console-based Python programs: a Password Strength Checker and an Email Validation System. Both use built-in string methods for validation and feature a common boxed-menu interface.

PROGRAM 1 — PASSWORD STRENGTH CHECKER

2.1 Introduction and Background

Weak passwords are a major security risk. This program develops a console-based Password Strength Checker in Python that evaluates passwords using predefined rules and provides clear feedback. It focuses on rule-based string validation and character analysis using Python's built-in string methods.

2.2 Objectives

- Develop a console-based Password Strength Checker.
- Validate password length, uppercase, lowercase, digits, and special characters.
- Classify passwords as Weak, Medium, or Strong.
- Display a strength bar and requirement checklist.
- Practice Python string validation methods.
- Test the program with different password combinations.

2.3 Problem Statement / Driving Question

How can a password be evaluated using simple rule-based checks to provide clear feedback and classify its strength?

2.4 Methodology

Program Structure: Used reusable box-drawing helper functions to create a consistent bordered menu and result interface with an ASCII-art banner.

Password Validation Logic: Developed password_validator() to check password length, digit, lowercase, uppercase, and special character requirements, then classified passwords as weak, medium, or strong.

Result Presentation: Displayed a strength bar, an **[OK]/[NO]** requirement checklist, and formatted feedback inside the bordered interface.

Menu-Driven Control Flow: Implemented a continuous menu with **Check Password** and **Exit** options, and tested the program with different password combinations to verify correct classification.

2.5 Expected Outcomes / Deliverables

- A functional Password Strength Checker.
- Password classification with visual feedback.
- A user-friendly boxed console interface.
- Practical understanding of rule-based string validation.

- Scope for future improvements such as entropy-based scoring and common-password detection.

3.6 Core Logic

The password_validator() function is the core of the Password Strength Checker. It validates the password against five security criteria and classifies it as Weak, Medium, or Strong based on the rules satisfied. This keeps the validation logic modular, readable, and easy to extend.

```
def password_validator(password):
    has_num = any(ch.isdigit() for ch in password)
    has_lower = any(ch.islower() for ch in password)
    has_upper = any(ch.isupper() for ch in password)
    has_special = any(ch in SPECIAL_CHARS for ch in password)
    long_enough = len(password) >= 8

    if not long_enough:
        return "weak", "Password needs to have at least 8 characters.", checks
    if has_num and has_lower and has_upper and has_special:
        return "strong", "Password is Strong! Good to go.", checks   # all criteria met
    elif has_num and has_lower:
        return "medium", "Try adding an uppercase letter and a special character.",
checks
```

Note: the strength categories currently rely on fixed rule combinations rather than a weighted score; introducing an entropy- or scoring-based model is planned as a future enhancement.

2.7 Output Screenshot

PROGRAM 2 — EMAIL VALIDATION SYSTEM

3.1 Introduction and Background

Email validation is an essential part of many software applications. This program develops a console-based Email Validation System in Python that checks an email address using structural rules and provides clear validation feedback. It applies character-level string analysis using Python's built-in string methods without regular expressions.

3.2 Objectives

- Design and implement a console-based Email Validation System.
- Check for a single '@' symbol and absence of spaces.
- Validate the local part and domain separately.
- Verify the domain format and top-level domain (TLD).
- Accept only predefined common TLDs.
- Test the validator with valid and invalid email addresses.

3.3 Problem Statement / Driving Question

How can an email address be validated using simple rule-based checks while providing clear and specific error messages for invalid formats?

3.4 Methodology

Program Structure: Reused the box-drawing helper functions to create a consistent bordered menu and result interface with an ASCII-art banner.

Local Part and Domain Validation: Developed name_check() to validate the local part and domain_check() to verify the domain structure and TLD.

Overall Email Validation Logic: Implemented email_checker() to perform basic format checks, validate the local and domain parts, and verify the TLD against a predefined list.

Menu-Driven Control Flow: Built a continuous menu with **Check Email** and **Exit** options, and tested the validator with different valid and invalid email formats.

3.5 Expected Outcomes / Deliverables

- A functional Email Validation System.
- Clear validation messages for invalid email formats.
- A user-friendly boxed console interface.
- Practical understanding of rule-based string validation.
- Scope for future improvements such as regex support and an expandable TLD list.

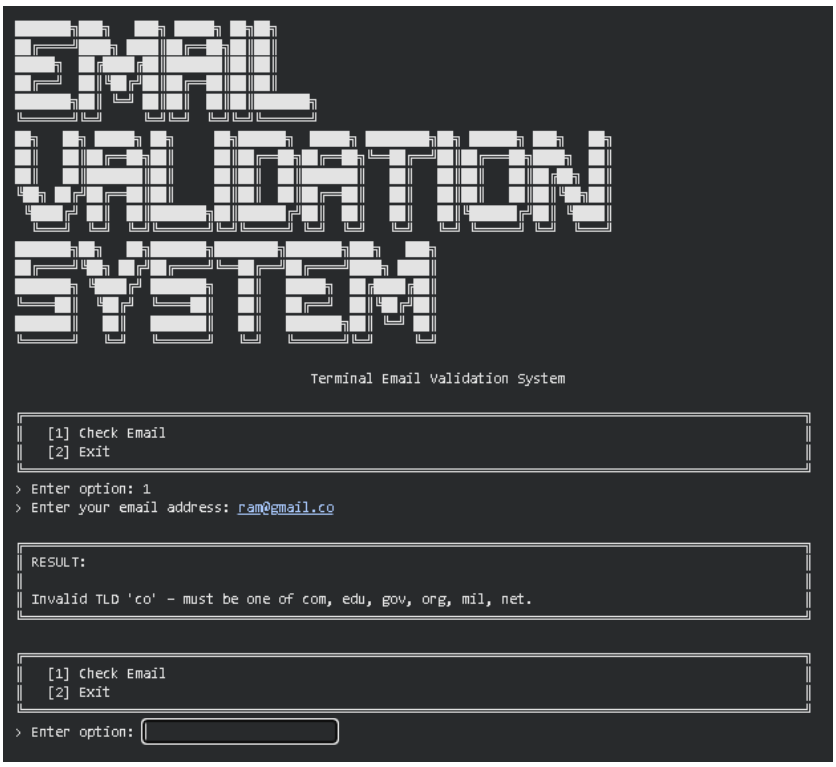
3.6 Core Logic

The domain_check() function is the core of the Email Validation System. It validates the domain format by checking for a single dot, non-empty alphanumeric domain and TLD parts, and a valid TLD length. This modular approach makes the validation logic simple, reusable, and easy to maintain.

```
def domain_check(domain):
    # Must have exactly one dot, and only letters/digits before and after
    if domain.count('.') != 1:
        return False
    local, tld = domain.split('.')
    if not local or not tld:
        return False
    if not local.isalnum() or not tld.isalnum():
        return False  # reject symbols/spaces in domain parts
    if len(tld) < 2:
        return False  # TLD must be at least 2 characters
    return True
```

Note: the system currently restricts valid emails to a fixed TLD set and alphanumeric-only local/domain parts, so addresses using hyphens, subdomains, or newer TLDs are rejected; broadening this coverage is planned as a future enhancement.

3.7 Output Screenshot



4. Tools & Technologies Used

Tool / Technology	Purpose
Python 3.x	Core programming language for both programs
String Methods (`isdigit`, `islower`, `isupper`, `isalnum`, `split`, `count`)	Password and email validation
Python Dictionaries	Store password validation results
Python Sets	Store valid TLDs for email validation
f-Strings	Format output and validation messages
Functions & Control Flow	Organize validation logic and program flow

5. References

- Python Official Documentation: https://docs.python.org/3/
- Python String Methods Reference: https://docs.python.org/3/library/stdtypes.html#string-methods
- OWASP Password Strength Guidance: https://owasp.org/www-community/controls/Password_strength
- IANA List of Top-Level Domains: https://www.iana.org/domains/root/db
- Brainware University - Python Programming Course Guidelines

6. Signatures

Signature of Student(s): **Date of Submission:**

Signature of Guide/Supervisor: